



北京邮电大学

Beijing University of Posts and Telecommunications

第九讲 强化学习

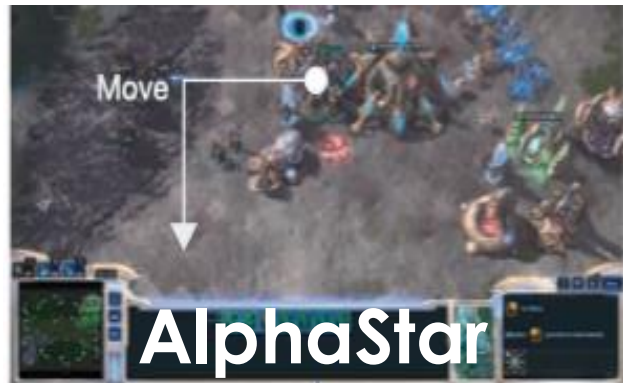
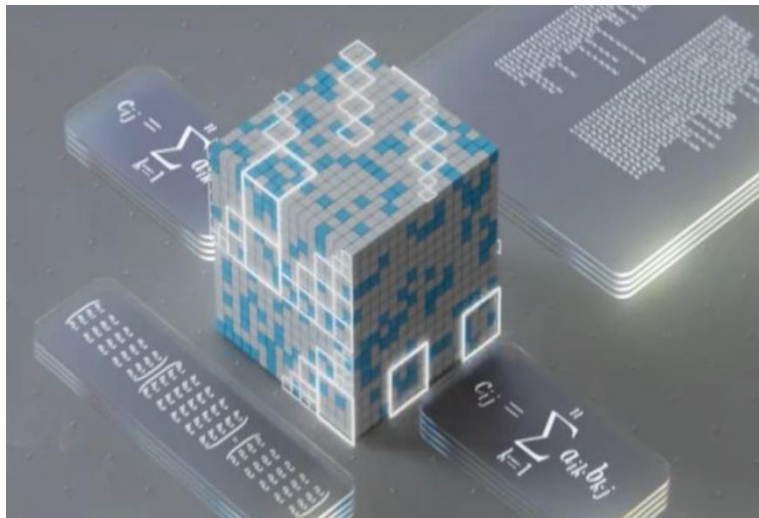
戚 琦

网络与交换技术国家重点实验室 网络智能研究中心 科研楼511

qiqi8266@bupt.edu.cn

13466759972





AlphaTensor





- 强化学习案例
- 强化学习基本思想
- 马尔科夫决策过程
- 贝尔曼方程
- 强化学习算法



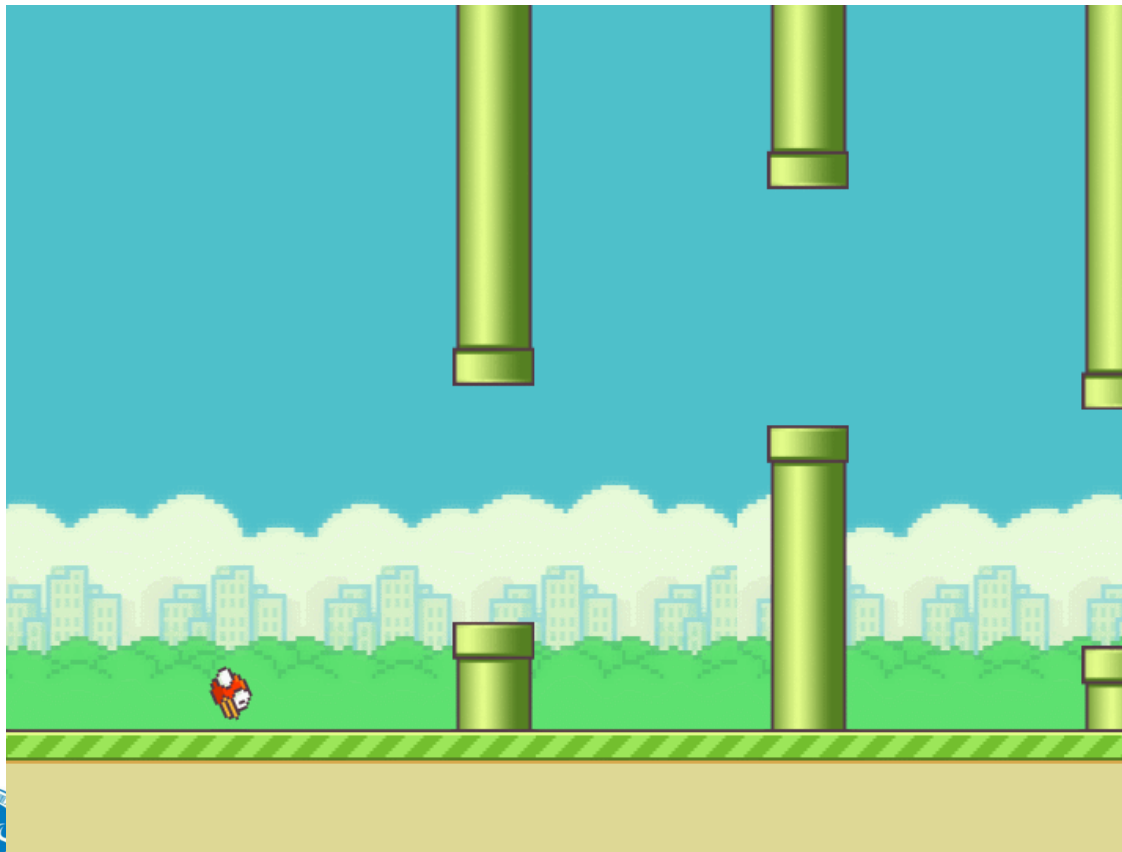
试错：尝试不同的做法（**动作**）

强化：有意义多做，没有意义少做

目标：是努力的方向，目标导向

学习者（人或者动物）**必须尝试**各种动作，并且渐渐**趋近于**那些表现最好的动作，以达到**目标**。

勇于尝试，向更好的目标前进！

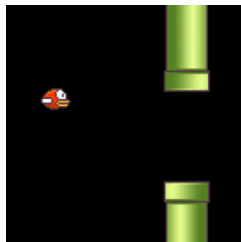


Flappy Bird

<https://www.bilibili.com/video/BV14741187Ar?from=search&seid=13360919136365850002>



更多案例



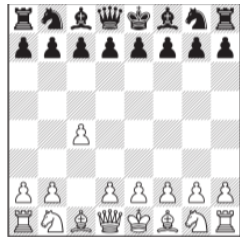
观察场景



游戏动作



游戏得分



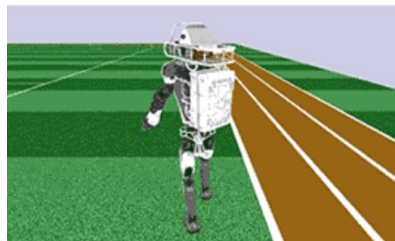
看到态势



走一步棋



输赢



观察环境



迈出一步



走到头



观察魔方



转一下



魔方复原



监督学习



"Hello"

Say "Hi"



"Bye bye"

Say "Good bye"

强化学习



.....



.....

.....

Hello 😊

Agent

.....

Agent



Bad



对比思考

有标签的数据



监督学习:希望模型根据指定的输入得到对应的输出

没有具体的标签,
但是环境有反馈



强化学习:希望模型根据指定的状态得到使回报最大化的行动

- 监督学习以明确的样本标签作为经验数据或先验知识直接告诉模型该如何完成指定任务
- 强化学习主要通过学习先验知识寻求最优决策。

强化学习使用的经验数据或先验知识则较为模糊，通常是由智能体所处环境提供的某种反馈信息。



- 强化学习案例
- **强化学习基本思想**
- 马尔科夫决策过程
- 贝尔曼方程
- 强化学习算法



1951年Minsky提出强化学习

五大要素

环境

状态



动作

前、后、
左、右

奖赏

累计得分 100

智能体



智能体从环境中获得状态

目标驱动选择执行一个动作

动作作用到环境，得到奖励

旧环境->新环境

学习时与环境交互

有一个清晰的目标



智能体：学习的主体

- 能感知环境的状态
- 选择动作（可能会影响环境的状态）
- 目标导向（与环境状态有关的）
- 长期收益

动作：智能体发出的行为

环境：智能体所处的外部环境，也是智能体的交互对象

状态：智能体当前所处的可观察状态

- **策略**定义了智能体在给定时间内的行为方式
 - 一个**策略**就是从环境感知的状态到在这些状态中可采用动作的一个**映射**
 - 可能是一个**查找表**，也可能是一个**函数**
-
- **奖励**函数是强化学习问题中的**目标**，它把环境中感知到的状态映射为单独的一个奖赏
 - 奖励函数是改变策略的基础

强化学习的**目标**是通过机器学习方式有效解决**序贯决策问题**，或者说通过机器学习方式实现**对连续多步自动决策问题的优化求解**。

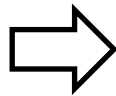
序贯决策过程完成任务时需要连续进行多步决策的过程；

序贯决策问题是如何让计算机像人类一样能够自动进行合理的序贯决策。

- 关注**试错学习**，以心理学为起点（贯穿人工智能早期）
- 关注**最优控制**问题以及使用值函数和动态规划的解决方案

(马尔科夫决策过程为理论)

以最终目标为导向，把不同的子任务整合到一个大框架中

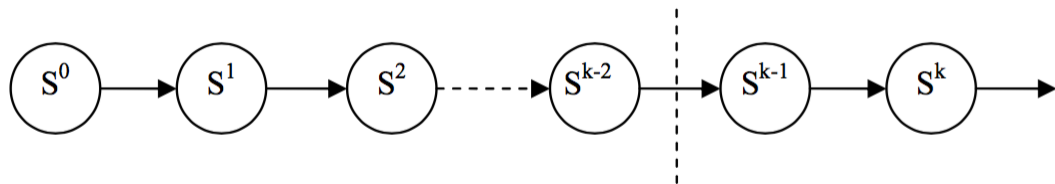


1. 系统有连续的状态
例如，连续的业务请求、位置移动、数据包发送、连续的视频...
2. 有选择判决的需求
3. 每个决策对应着状态的变化

- 强化学习案例
- 强化学习基本思想
- 马尔可夫决策过程
- 贝尔曼方程
- 强化学习算法



马尔可夫过程



问题被划分为多个阶段

在过程或系统在时刻 t_0 所处的状态为已知的条件下，过程在时刻 $t > t_0$ 所处状态的条件分布与过程在时刻 t_0 之前所处的状态无关的特性称为

马尔可夫性或无后效性

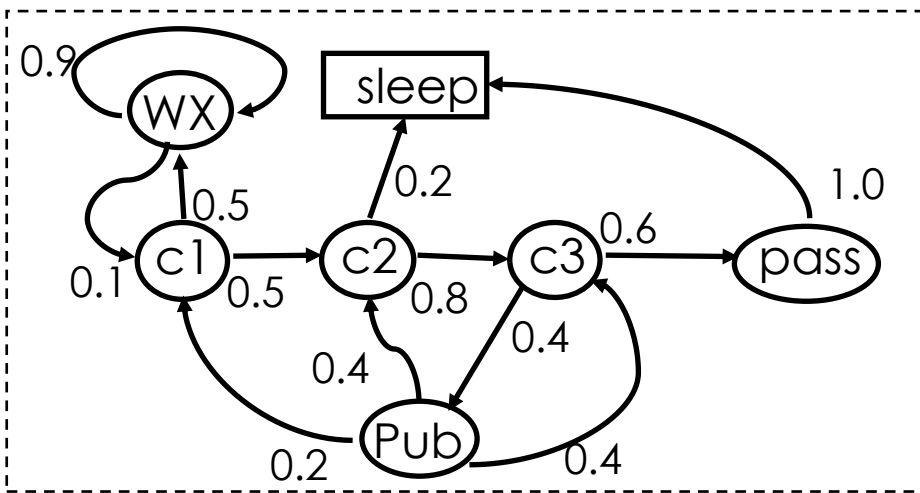
无后效性：未来与过去无关

S: 状态

机器人看到的环境
游戏屏幕、棋盘
机械手的姿态
网约车的位置
...

马尔可夫决策过程与马尔可夫过程的本质区别：**智能体即决策者的介入，增加了决策动作集A，由智能体触发状态转移**

状态之间有转移概率

$$P = \begin{matrix} & \begin{matrix} C1 & C2 & C3 & Pass & Pub & WX & Sleep \end{matrix} \\ \begin{matrix} C1 \\ C2 \\ C3 \\ Pass \\ Pub \\ WX \\ Sleep \end{matrix} & \begin{bmatrix} & & & & & 0.5 & \\ & 0.5 & & & & & \\ & & 0.8 & & & & 0.2 \\ & & & 0.6 & 0.4 & & 1.0 \\ 0.2 & 0.4 & 0.4 & & & & \\ 0.1 & & & & & 0.9 & \\ & & & & & & 1 \end{bmatrix} \end{matrix}$$


称条件概率 $P_{ij}(m, m+n) = P\{X_{m+n} = s_j | X_m = s_i\}$ 为马氏过程
在时刻 m 处于状态 s_i 条件下, 在时刻 $m+n$ 转移到状态 s_j 的转移概率
由转移概率组成的矩阵称为**转移概率矩阵**, $P(m, m+n) \triangleq (P_{ij}(m, m+n))$

转移概率矩阵的每一行元素之和等于1.

$$\sum_{j=1}^{\infty} P_{ij}(m, m+n) = 1, i = 1, 2, \dots$$

- 用状态转移描述系统的动态性
- 马尔可夫**决策**过程（MDP, Markov decision processes）：
是基于马尔可夫过程理论的随机动态系统的**最优决策过程**。
- **可周期或连续**观察的随机动态系统的**最优化问题**。
- 各个时刻根据观察到的状态，从**决策集合**中选用一个决策，**确定系统下次的转移概率**与相应的运行**效果**。
- 假设这两者都不依赖于系统过去的历史，在各个时刻选取决策的目的是使系统运行的全过程达到某种最优运行效果，即**选取控制（影响）系统发展的最优策略**。

- 马尔可夫决策过程（MDP）最基本模型是一个**四元组** $\langle S, A, T, R \rangle$

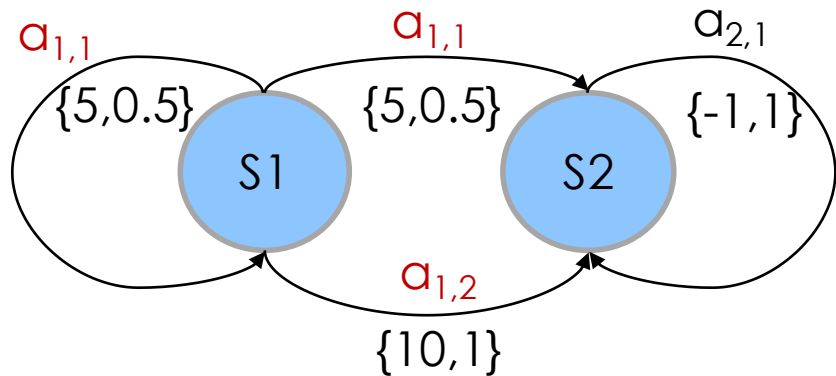
分别是状态空间、动作集合、转移概率和期望奖励

$$\{S; A(i) | i \in S; p_{ij}(a) | i, j \in S, a \in A(i); r(i, a) | i \in S, a \in A(i)\}$$

- **状态空间**：每个阶段开始时系统所有可能的状态记为集合 $S=\{1,2,3,\dots\}$ ， S 被称为系统的**状态空间**。
- **动作集合**： $A(i)$ 表示在某阶段系统状态为 i 时可供选择的有限个可行**动作集合**。
- **转移概率**：假设某阶段，系统处于状态 $i \in S$ ，决策为 $a \in A(i)$ ，则下一阶段的状态为 $j \in S$ 的**转移概率**为 $p_{ij}(a)$ 。
- **期望奖励**：系统在某阶段处于状态 i ，决策为 $a \in A(i)$ ，所获得的**期望奖励**记为 $r(i, a)$



示例---两状态MDP



转移概率 Transition Probabilities:

$$\begin{aligned}
 p_t(s_1 | s_1, a_{1,1}) &= 0.5, & p_t(s_2 | s_1, a_{1,1}) &= 0.5, \\
 p_t(s_1 | s_1, a_{1,2}) &= 0, & p_t(s_2 | s_1, a_{1,2}) &= 1, \\
 p_t(s_1 | s_2, a_{2,1}) &= 0, & p_t(s_2 | s_2, a_{2,1}) &= 1,
 \end{aligned}$$

判决时刻: $T = \{1, 2, \dots, N\}$

状态空间 States: $S = \{S1, S2\}$

动作集合 Actions :

$$A_{s1} = \{a_{1,1}, a_{1,2}\} \quad A_{s2} = \{a_{2,1}\}$$

奖励函数:

$r_t(s, a, j)$ 表示时刻 t , 状态 s , 执行动作 a , 在 $t+1$ 转移到状态 j 的收益

Rewards: $r_t(s_1, a_{1,1}) = 5$ $r_t(s_1, a_{1,2}) = 10$ $r_t(s_2, a_{2,1}) = -1$



示例---两状态MDP

假设需要在时刻1和时刻2判决:

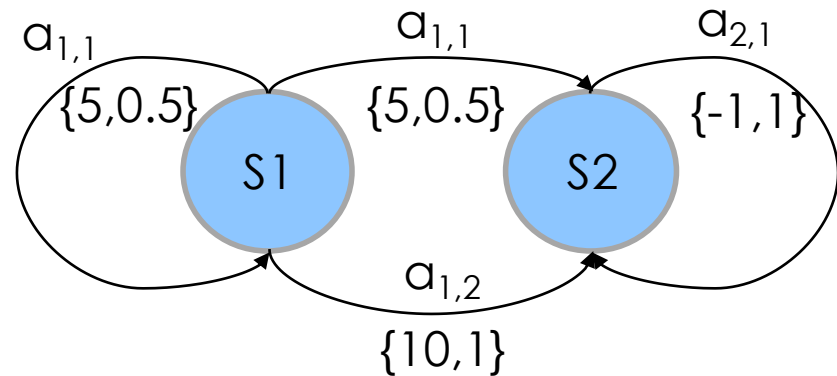
判决点1: $d_1(s_1) = a_{1,1}$, $d_1(s_2) = a_{2,1}$

判决点2: $d_1(s_1) = a_{1,2}$, $d_1(s_2) = a_{2,1}$

相应的收益:

$$r_1(s_1, d_1(s_1)) = 5, \quad r_1(s_2, d_1(s_2)) = -1$$

$$r_2(s_1, d_2(s_1)) = 10, \quad r_2(s_2, d_2(s_2)) = -1$$



总收益: 如果初始状态是 s_1 :

$$r_1(s_1, a_{1,1}) + p_1(s_1|s_1, a_{1,1}) r_1(s_1, a_{1,2}) + p_1(s_2|s_1, a_{1,1}) r_2(s_2, a_{2,1}) = 9.5$$

↑
0.5

↑
0.5

- 根据多阶段的特征，马尔可夫决策分为有限阶段马尔可夫决策和无限阶段马尔可夫决策。
- 有限阶段马尔可夫决策：解决 $T(T < \infty)$ 个阶段内的期望奖励最大化（或期望成本最小化）问题。 T 为阶段长度。有限阶段马尔可夫决策规划实际上是一种随机动态规划方法。
- 无限阶段马尔可夫决策：当面临的问题阶段很长，并且难以确定阶段究竟有多长时，比较简便的做法是假定阶段长度是无限的，即 $T = \infty$

赢得目标的关键->得到好的策略->最大化长期回报（奖赏）

环境可以提供我们某一时刻 t 的奖赏(立即回报) r

采取某一个行动希望最大化长期回报 $maximize \sum_{k=0} r_{t+k+1}$

★ 无限阶段的问题：计算时间（若游戏是无限循环，则无法计算）

解决方法：降低未来回报对当前的影响，也就是对未来的奖励乘以一个折扣率，这个奖励就是有限的。

长期奖励：将当前状态之后的所有回报取出来，分别乘以对应的打折率，然后加起来得到的一个汇总值，

$$\sum_{k=0} \gamma^k r_{t+k+1}$$

在阶段0, 1, ...时以一定的概率获得一串回报，累加后就是效用函数（目标函数），引入0-1之间的折扣因子 γ

折扣因子的另一个含义是人为认定过程在每步执行都有较小的非零概率 $1 - \gamma$ 终止，例如“会话结束”

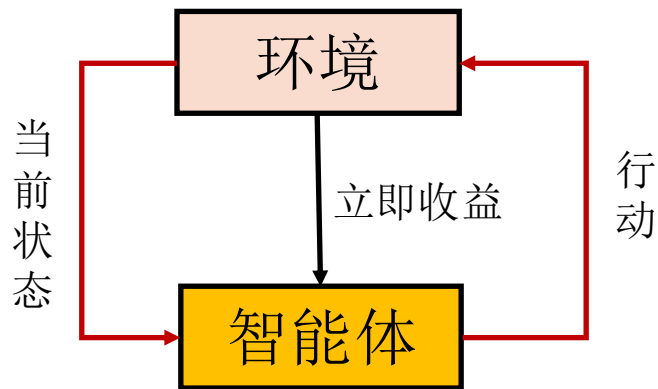
γ^k 随着时间推移， k 变大，这个值会变小。
时刻1的收益，略小于时刻0；
时刻2的收益略小于时刻1；...
保证收敛



小结：马尔可夫决策模型

■ 适用系统的三大特点：

- 状态转移的无后效性
- 状态转移有不确定性
- 智能体所处的每步状态完全可以观察



自动驾驶

对话系统

网络控制

系统建模

分析问题

设计算法

求解问题

马尔可夫决策模型





- 强化学习案例
- 强化学习基本思想
- 马尔科夫决策过程
- 贝尔曼方程
- 强化学习算法



■ 贝尔曼提出了强化学习算法求解的基石：贝尔曼方程

每个状态 s 下，有多个动作 a 选择，每执行一个动作，系统转移到另一个状态

$\pi(s)$ ：策略函数，输入 s 输出 a

价值函数：以期望形式定义

{ “状态”价值函数（state value function）
“状态-动作”价值函数（state-action value function）

强化学习的核心问题就是求最优化策略函数



理查德-贝尔曼
(Richard Bellman)，
美国数学家，美国全
国科学院院士，动态
规划的创始人

“状态”价值函数的含义：从当前状态开始到最终状态时所获得的累加回报的期望，即**评估一个状态的收益**

从不同的状态出发，得到的值可能不同，从同一状态出发使用不同的策略，最后的值也可能不一样。

“状态”价值函数定义： $V^\pi(s) = E_\pi [R_t | s_t = s]$ $R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$

改写为递归形式

$$\begin{aligned}
 V^\pi(s) &= E_\pi [R_t | s_t = s] \\
 &= E_\pi \left\{ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s \right\} \\
 &= E_\pi \left\{ r_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k r_{t+k+2} \mid s_t = s \right\} \\
 &= \sum_a \pi(s, a) \sum_{s'} P_{ss'}^a \left[\underline{R_{ss'}^a} + \gamma \underline{E_\pi \left\{ \sum_{k=0}^{\infty} \gamma^k r_{t+k+2} \mid s_{t+1} = s' \right\}} \right] \\
 &= \sum_a \pi(s, a) \sum_{s'} \cancel{P_{ss'}^a} \left[\underline{R_{ss'}^a} + \gamma \underline{V^\pi(s')} \right]
 \end{aligned}$$

选择动作a的概率

状态s转移到s'的概率

状态s'的价值

“状态-动作”价值函数的含义：给定一个状态 s_t ，采取动作 a_t 后，按照某一策略 π_s 与环境继续进行交互，得到的累计回报的期望值。也称为Q函数，是对（状态-动作对）的评估。

“状态-动作”价值函数定义 $Q^\pi(s, a) = E_\pi [R_t | s_t = s, a_t = a]$

$$Q^\pi(s, a) = E_\pi \{R_t | s_t = s, a_t = a\} = E_\pi \left\{ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | s_t = s, a_t = a \right\}$$

两种价值函数关系： $V^\pi(s) = \sum_{a \in A} \pi(a|s) * Q^\pi(s, a)$ 动作不确定，考虑所有动作

$$Q^\pi(s, a) = R_s^a + \gamma * \sum_{s'} P_{ss'} V^\pi(s) \quad \text{考虑特定动作}$$

求解最优策略

最大化“状态”价值函数：求当前状态的最大期望值

$$\begin{aligned} V^*(s) &= \max_a E \{ r_{t+1} + \gamma V^*(s_{t+1}) | s_t = s, a_t = a \} \\ &= \max_a \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^*(s')] \end{aligned}$$

状态价值函数中，当前状态下选取哪个动作是未知数，需要求出一系列的动作集合（各不同状态下），形成一个完整的策略，然后使状态方程的值最大化

最大化“状态-动作”价值函数：求当前状态下一某个动作能带来的最大回报值

$$\begin{aligned} Q^*(s, a) &= E \left\{ r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') | s_t = s, a_t = a \right\} \\ &= \sum_{s'} P_{ss'}^a \left[R_{ss'}^a + \gamma \max_{a'} Q^*(s', a') \right] \end{aligned}$$

动作价值函数是当前状态下的动作已知，求余下状态下的动作集合使动作方程的值最大化

讨论：如何求解最优策略

- 学习者**必须尝试**各种动作，并且渐渐**趋近于**那些表现最好的动作，以达到**目标**。
- 尝试各种动作即为试错，也称为**探索**（ **exploration** ）
- 趋近于好的动作即为强化，也称为**利用**（ **exploitation** ）

探索与利用之间的平衡，是强化学习的一个挑战。

探索多了，有可能找到差的动作，探索少了，有可能错过好的动作。

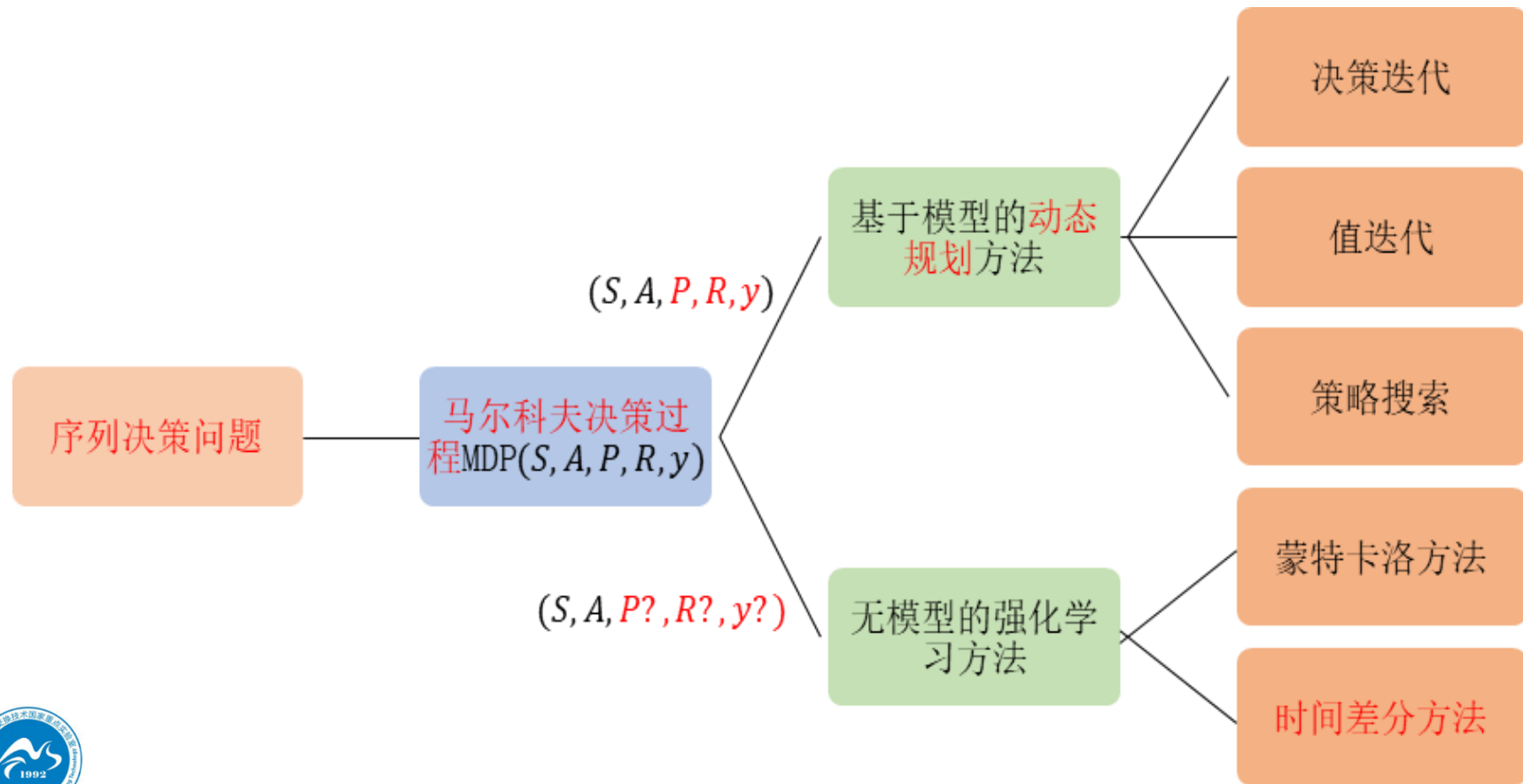
监督学习中根本就不存在探索和利用的平衡问题，样例学习阶段之后，就只有利用了。

随时间推移， 学习者利用它获得的经验**不断**提高自己的性能。

强化学习就是**试出来的经验！**



强化学习算法



- 求解最优策略最直接的手段就是**动态规划算法**，通过求解子问题的最优解得到整个问题的最优解

优化目标：寻找一个策略使得“状态”价值函数最大化

$$\pi^*(s) = \arg \max_{\pi} V_{\pi}(s)$$

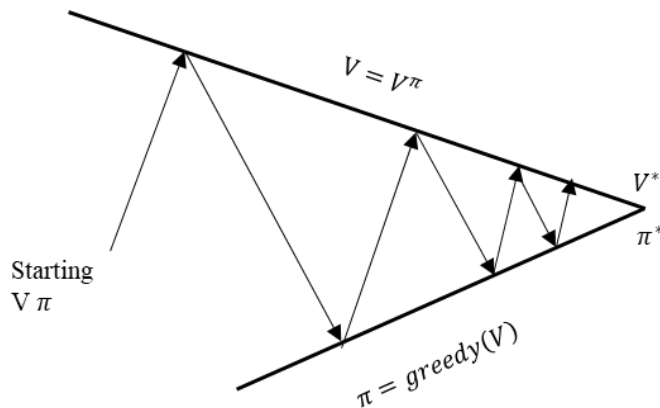
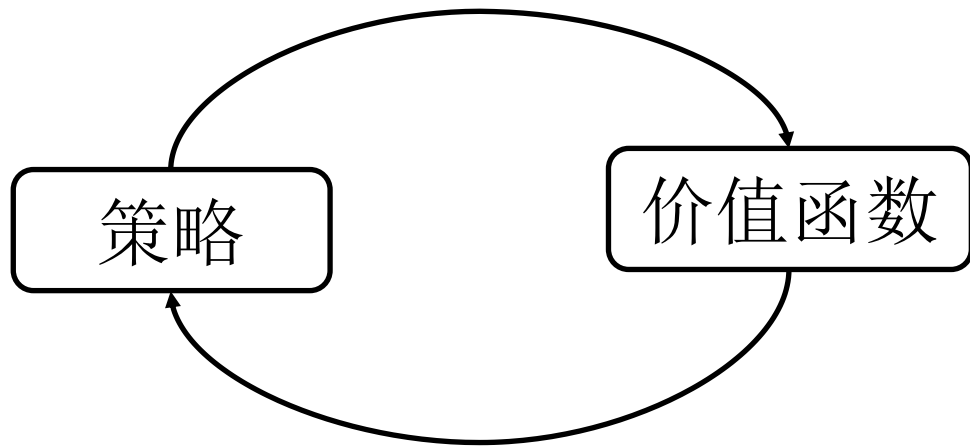
最优“状态”价值函数和最优“状态-动作”价值函数之间的关系：

$$V^*(s) = \max_a Q^*(s, a)$$

要保证“状态”价值函数最优，则当前动作要最优

算法要寻找“状态”价值函数的最大策略，需要**确定一个策略的“状态”价值函数**，通过调整策略，让其变大。用分步迭代方法，让其改进，分为**策略评估**与**策略改进**。

策略评估：计算“状态”价值函数



策略改进：通过某种规则调整当前策略

对所有状态 s ，将策略函数 $\pi(s)$ 和“状态”价值函数 $V(s)$ 初始化为任意值

第一阶段：策略估计

For

初始化相邻两次迭代的价值函数的差值： $\Delta = 0$

For $s \in S$ // 所有状态遍历

获取上次迭代的状态 s 的价值函数： $v = V(s)$

更新状态 s 的价值函数： $V^\pi(s) = \sum_a \pi(s, a) \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^\pi(s')]$

更新差值： $\Delta = \max(\Delta, |v - V(s)|)$

End $\Delta < \theta$

θ 是人工设置的阈值，判断“状态”价值函数的估计值是否收敛

第二阶段：策略改进

初始化策略是否改进的标志变量 $\text{flag} = \text{true}$

For $s \in S$

获取之前的策略所确定的动作: $a = \pi(s)$

计算当前状态的最优动作: $\pi(s) = \arg \max_a V(s)$

如果 $a \neq \pi(s)$, 则 $\text{flag} = \text{false}$

如果 **$\text{flag} = \text{true}$** 则停止迭代, 返回 π 和 V ; 否则继续执行策略估计

- 策略评估计算量大，需要多次处理所有状态，不断迭代更新态价值函数
- 实际上不需要知道状态价值函数的精确值
- 整体最优解，其局部也是最优
- 最优策略分为两个部分

状态 S 到 S' 采用最优策略

状态 S' 采用的策略也是最优

直接收敛到最优的价值函数！

初始化所有状态的价值函数为任意值

For

初始化相邻两次迭代时价值函数的最大差值: $\Delta = 0$

For $s \in S$

获取迭代之前的状态的价值函数: $v = V(s)$

更新状态 s 的价值函数:

$$V(s) = \max_a \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^*(s')]$$

更新差值: $\Delta = \max(\Delta, |v - V(s)|)$

End $\Delta < \theta$

迭代结束后, 输出一个确定性的策略 π , 每个状态下选择价值最大的动作:

$$\pi(s) = \arg \max_a \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^*(s')]$$

回顾策略迭代中:

更新状态 s 的价值函数:

$$V^\pi(s) = \sum_a \pi(s, a) \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^\pi(s')]$$

无模型强化学习：无需状态转移概率与立即奖励

无模型强化学习基本思想也是策略评估与策略改进

■ 蒙特卡洛算法：样本是一些episode（片段）

从某个一状态开始，执行一些动作，直到终止状态为止的一个完整的状态和动作序列。

从样本**Episode**中学习，估算“状态”价值函数和“状态-动作”价值函数
没有终止状态的情况则无法处理

■ 时序差分学习（TD）：执行一个动作后就进行价值函数估计

无需完整Episode，不依赖转移概率

随机产生样本计算 $V(s)$



Q-learning是经典的**价值迭代**算法：估计“状态-动作”价值函数的最大值，通过迭代找到价值函数最优值，从而确定最优策略

初始化：将所有非终止状态的 $Q(s,a)$ 初始化为任意值，终止状态的初始化为0

For all episode

选择一个初始值状态 s

For each step

根据Q函数为状态 s 确定一个动作 a ，可以**采用 ϵ 策略**

执行动作 a ，得到立即奖励 R 以及下一个状态 s'

更新“状态-动作”价值函数：

$$Q(s, a) = Q(s, a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

更新差值

进入新状态： $s=s'$

End s 为终止状态

End

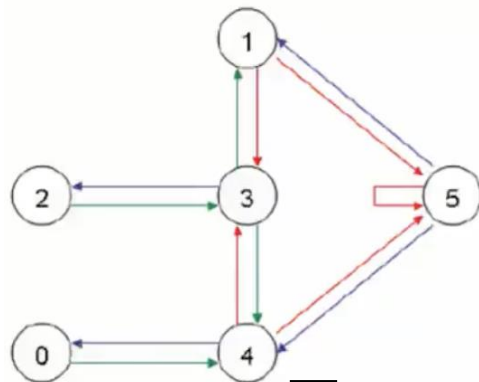
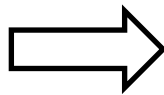
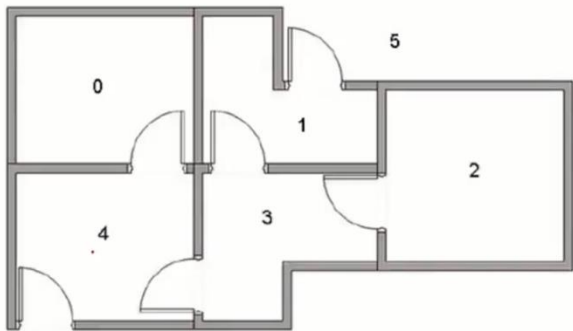
学习率

s 状态下执行动作 a 获得的奖励

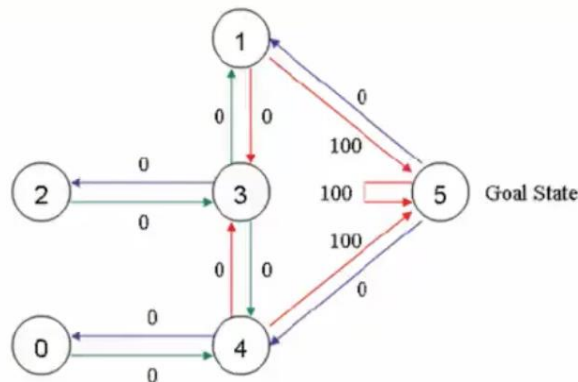
ϵ 贪心策略，以 $1 - \epsilon$ 概率执行**Q值的最大动作**；以 ϵ 随机选择动作



最简单的例子



设置奖励

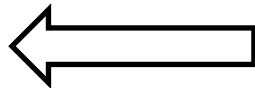


环境 (s_1, s_2, s_3, s_4, s_5)

目标：走到状态5出迷宫

State	Action					
	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

写成奖励矩阵R

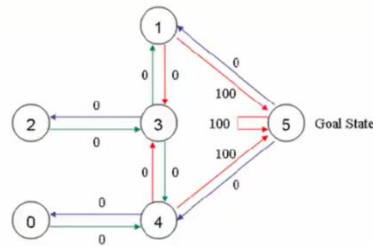


-1是填充，
达成目标是100

最简单的例子

设置学习率0.8，初始状态为 s_1

初始Q Table，行为状态，列为动作



$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

更新

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

例如，先填写 $Q(1,5)$ ，当位于状态5时，可能转到1、4、5

$$\begin{aligned} Q(1,5) &= R(1,5) + 0.8 * \max\{Q(5,1), Q(5,4), Q(5,5)\} \\ &= 100 + 0.8 * \max\{0, 0, 0\} \\ &= 100. \end{aligned}$$

这一次episode结束。

再随机选择状态3(s)作为初始状态，可能的动作对应转到1、2、4，选择转到状态1(a)，R矩阵中，状态1对应的行为可能转到3或5，再Q Table中找到 $Q(1,3)$ 和 $Q(1,5)$ ，计算 $Q(3,1)$

$$\begin{aligned} Q(3,1) &= R(3,1) + 0.8 * \max\{Q(1,3), Q(1,5)\} \\ &= 0 + 0.8 * \max\{0, 100\} \\ &= 80. \end{aligned}$$



最简单的例子

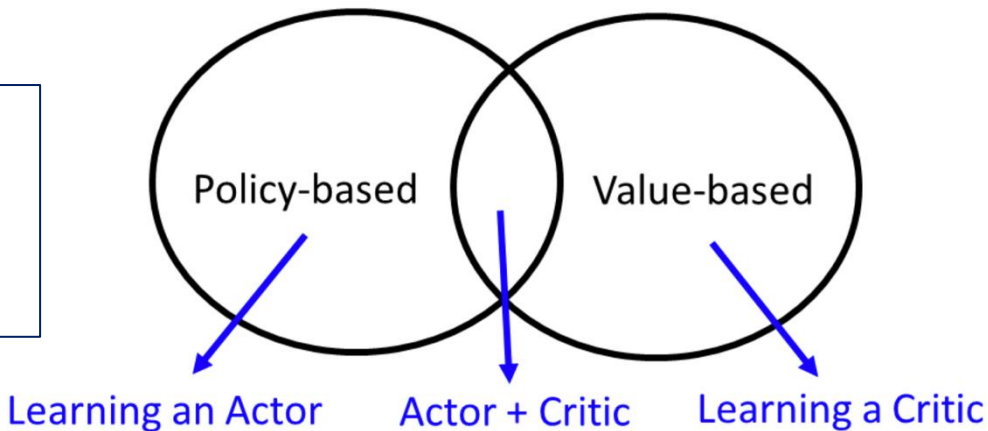
再次更新

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 80 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

若我们继续执行更多的 episode, 矩阵 Q 将最终收敛成

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 400 & 0 \\ 0 & 0 & 0 & 320 & 0 & 500 \\ 0 & 0 & 0 & 320 & 0 & 0 \\ 0 & 400 & 256 & 0 & 400 & 0 \\ 320 & 0 & 0 & 320 & 0 & 500 \\ 0 & 400 & 0 & 0 & 400 & 500 \end{bmatrix} \end{matrix} \longrightarrow Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 64 & 0 & 100 \\ 0 & 0 & 0 & 64 & 0 & 0 \\ 0 & 80 & 51 & 0 & 80 & 0 \\ 64 & 0 & 0 & 64 & 0 & 100 \\ 0 & 80 & 0 & 0 & 80 & 100 \end{bmatrix}$$

强化学习与
深度神经网络
同步发展



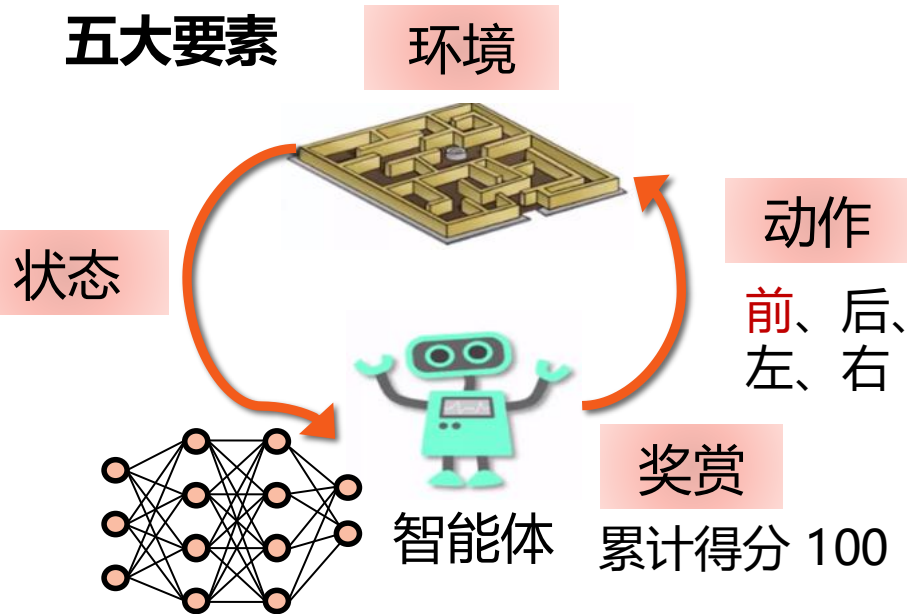
著名的Alpha Go是各种方法大杂烩，有Policy-based的方法，有Value-based的方法，有model-based的方法。

目前比较流行的深度强化学习方法是SAC、A3C、TRPO、ACKTR...



1951年Minsky提出强化学习

五大要素



智能体结构是深度神经网络，输入状态，输出策略，指导动作选择

动作作用到环境，得到奖励

旧环境 -> 新环境

2013年提出深度强化学习

基石论文: Playing Atari with **Deep Reinforcement Learning**. V. Mnih, K. Kavokcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller. **NIPS-DLW 2013**.



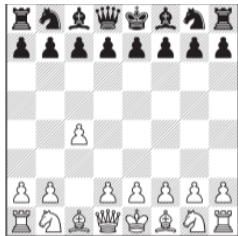
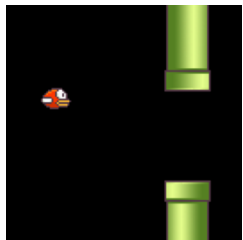
两大阵营的对垒



DeepMind

Google收购

游戏王者



NIPS 2013 Nature 2019.10 Science 2018

近200篇论文, 遍布NIPS, ICML,
ICLR, AAAI, IJCAI.....

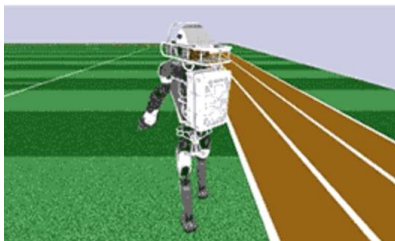
学术引领



OpenAI

马斯克牵头

机器人争霸



OpenGym

OpenAI Baselines

开源开放

众多方向的进展

ACKTR
(NIPS 2017)

TD3
(ICML 2018)

Multi-agent DRL
(ICML 2018, AAI 2018)

Transfer DRL
(ICML 2018)

Soft AC
(ICML 2018)

Variance Reduction
(ICLR 2018, ICML 2019)

Multi-task DRL
(ICML 2018)

Inverse DRL
(NIPS 2018)

优化自身性能

奖赏设计
M-level(IJCAI 2018)

监督信息
(KDD 2018)

更复杂的学习任务

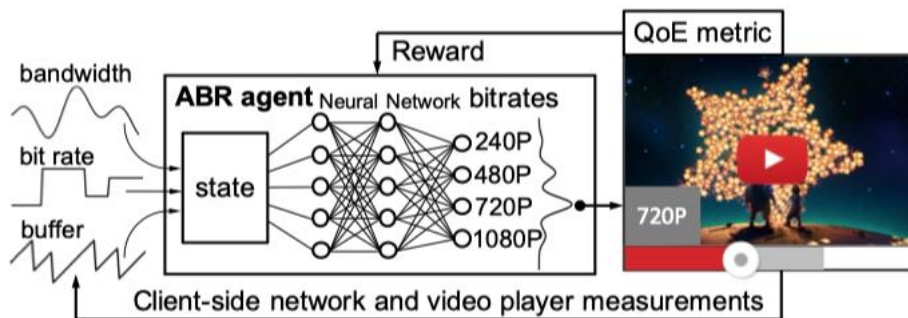
CNN, RNN, GCN, Attention, Capsule+DRL
(WWW 2019, KDD 2018, ICCV 2017, SIGCOMM)

解决具体场景问题

网约车派单



直播平台



提高专车司机的收入

- Agent: A driver 
- State: Discretized spatio-temporal grid
- Action: Pickup an order or do nothing
- Reward: Driver's income (order price)
- Episode: A whole day from 04:00 to 04:00 (+1)
- Value Function: The expected future income of a driver in a state S



让视频不卡顿

Playing Atari with Deep Reinforcement Learning

Volodymyr Mnih Koray Kavukcuoglu **David Silver** Alex Graves Ioannis Antonoglou

Daan Wierstra Martin Riedmiller

DeepMind Technologies

{vlad,koray,david,alex.graves,ioannis,daan,martin.riedmiller} @ deepmind.com

Abstract

We present the first deep learning model to successfully learn control policies directly from high-dimensional sensory input using reinforcement learning. The model is a convolutional neural network, trained with a variant of Q-learning, whose input is raw pixels and whose output is a value function estimating future rewards. We apply our method to seven Atari 2600 games from the Arcade Learning Environment, with no adjustment of the architecture or learning algorithm. We find that it outperforms all previous approaches on six of the games and surpasses a human expert on three of them.

DeepMind

NIPS WS 2013

CNN+Q-Learning

输入是原始像素点；输出是
值函数，估计未来的收益

大背景: Learning to control agents **directly from high-dimensional sensory inputs like vision and speech** is one of the long-standing challenges of reinforcement learning (RL).

小背景: Deep learningIt seems natural to ask whether similar techniques could also be beneficial for **RL** with sensory data.

问题: However reinforcement learning presents several challenges from a deep learning perspective.

四个challenge:

- 标注数据的问题
- 标量的信号
- 数据样本独立
- 数据分布随行为变化



This paper demonstrates that **a convolutional neural network** can overcome these challenges to learn successful control policies from raw video data in complex RL environments.

- 文章核心内容的概述，突出特色：

The network is trained with a variant of the **Q-learning** [26] algorithm, **with stochastic gradient descent** to update the weights. To alleviate the problems of correlated data and non-stationary distributions, we use **an experience replay mechanism** [13] which randomly samples previous transitions, and thereby smooths the training distribution over many past behaviors.

- 补充说明一些实验、效果、特色，突出主题

it learned from nothing but the video input, the reward and terminal signals, and the set of possible actions—**just as a human player would.**

Long-Ji Lin. Reinforcement learning for robots using neural networks. Technical report, DTIC Document, 1993.

- 主要介绍RL以及如何用
 - 是背景，不是作者的工作
 - 初学者可以用来自学

- Q-Learning $Q^*(s, a) = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q^*(s', a') \middle| s, a \right]$

长期收益

- function approximator

- **a non-linear function approximator: NN**

$$L_i(\theta_i) = \mathbb{E}_{s, a \sim \rho(\cdot)} \left[(y_i - Q(s, a; \theta_i))^2 \right]$$

y_u 是计算出来的真实值（标签）
Q是依赖于参数为 θ 的NN估计出来

损失函数为目标和预测值之间的方差

- RL-TD算法
- 详细分析最相关的文献：Perhaps the most similar prior work to our own approach is neural fitted Q-learning (NFQ) [20].
- [13]是RL+simple NN
- 作者不是第一个NN+RL，但是第一个端到端NN+RL
- Atari 2600 emulator

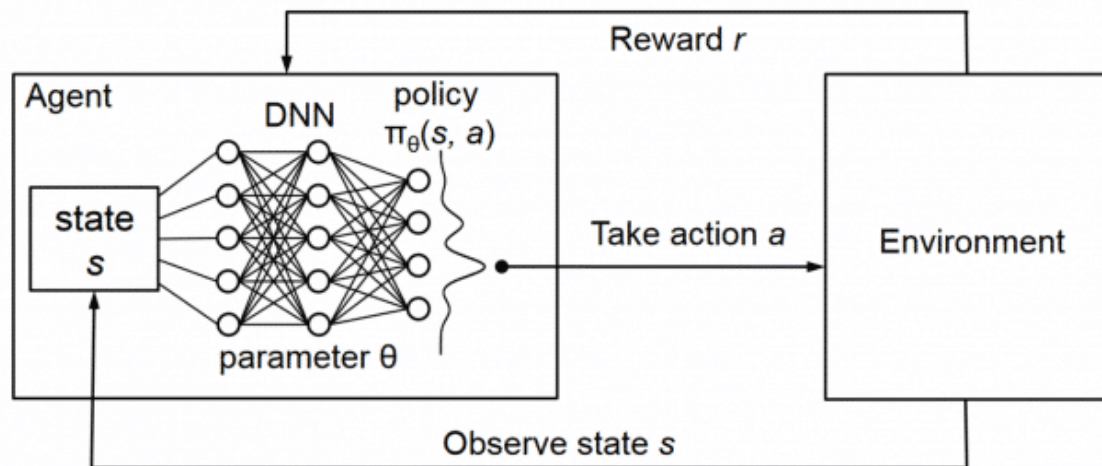
- *experience replay* (不断与环境交互中产生的数据)

- 把之前的环境、动作、收益

放到一个池子中，随机选 $e_t = (s_t, a_t, r_t, s_{t+1})$

分析：连续样本的相关性，非平稳分布，通过存储-采样打破

$\langle s_1, a_1, r_2, s_2 \rangle$
$\langle s_2, a_2, r_3, s_3 \rangle$
$\langle s_3, a_3, r_4, s_4 \rangle$
$\langle s_4, a_4, r_5, s_5 \rangle$
$\langle s_5, a_5, r_6, s_6 \rangle$
$\vdots$





Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N

Initialize action-value function Q with random weights

核心创新点

for episode = 1, M **do**

Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$

for $t = 1, T$ **do**

With probability ϵ select a random action a_t
otherwise select $a_t = \max_a Q^*(\phi(s_t), a, \theta)$

Q-learning是表格，这里用
函数（DNN表示Q）

存储

Execute action a_t in emulator and observe reward r_t and image x_{t+1}

Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$

采样

Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$

Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$

之前估计的目标Q值

Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3

end for

end for

目标值与真实值的误差





■ Preprocessing and Model Architecture

We now describe the exact architecture used for all seven Atari games. The input to the neural network consists is an $84 \times 84 \times 4$ image produced by ϕ . The first hidden layer convolves 16 8×8 filters with stride 4 with the input image and applies a rectifier nonlinearity [10, 18]. The second hidden layer convolves 32 4×4 filters with stride 2, again followed by a rectifier nonlinearity. The final hidden layer is fully-connected and consists of 256 rectifier units. The output layer is a fully-connected linear layer with a single output for each valid action. The number of valid actions varied between 4 and 18 on the games we considered. We refer to convolutional networks trained with our approach as Deep Q-Networks (DQN).



比较各算法在不同游戏中表现

■ 5.3 Main Evaluation

	B. Rider	Breakout	Enduro	Pong	Q*bert	Seaquest	S. Invaders
Random	354	1.2	0	-20.4	157	110	179
Sarsa [3]	996	5.2	129	-19	614	665	271
Contingency [4]	1743	6	159	-17	960	723	268
DQN	4092	168	470	20	1952	1705	581
Human	7456	31	368	-3	18900	28010	3690
HNeat Best [8]	3616	52	106	19	1800	920	1720
HNeat Pixel [8]	1332	4	91	-16	1325	800	1145
DQN Best	5184	225	661	21	4500	1740	1075

The games Q*bert, Seaquest, Space Invaders, on which we are far from human performance, are more challenging because they require the network to **find a strategy that extends over long time scales.**

Neural Adaptive Video Streaming with Pensieve

Hongzi Mao, Ravi Netravali, Mohammad Alizadeh
MIT Computer Science and Artificial Intelligence Laboratory
{hongzi,ravinet,alizadeh}@mit.edu



DRL解决码率
自适应问题

Pensieve: 冥想盆

背景：媒体播放中的码率控制

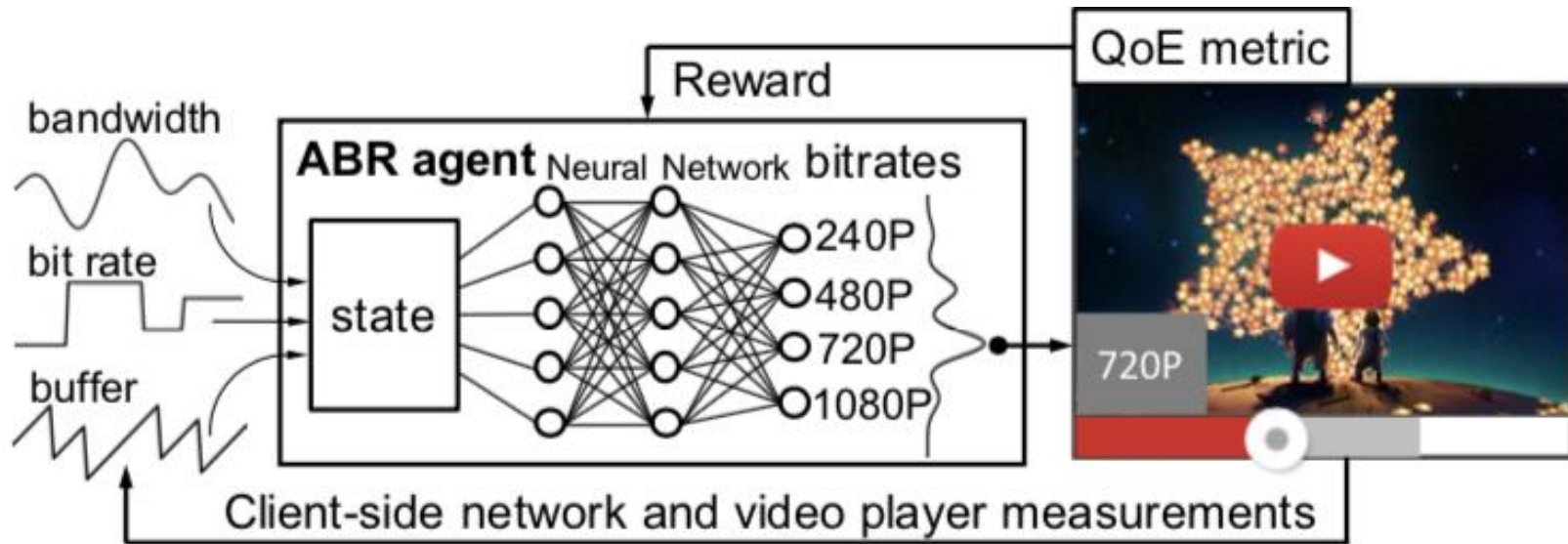


Figure 2: Applying reinforcement learning to bitrate adaptation.

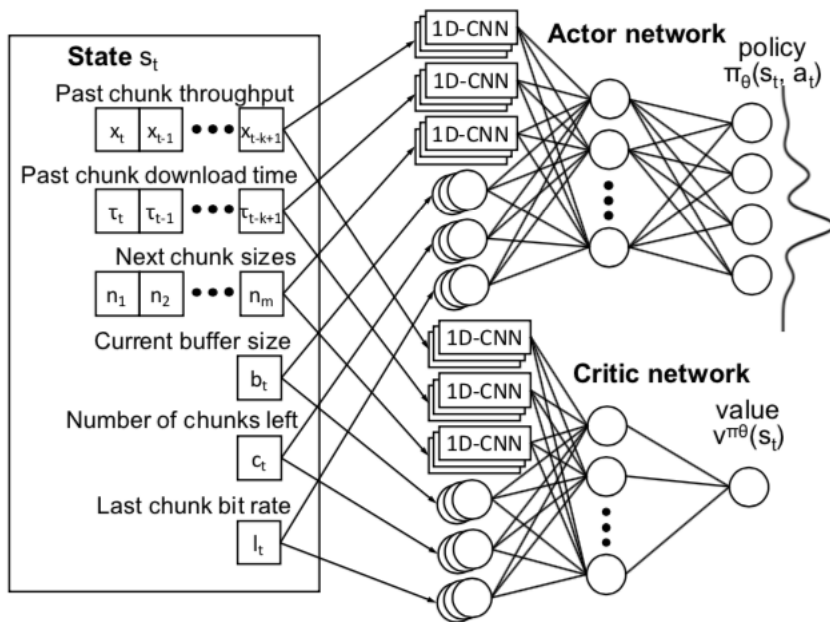


Figure 5: The Actor-Critic algorithm that Pensieve uses to generate ABR policies (described in §4.4).

采用AC算法，进行改进

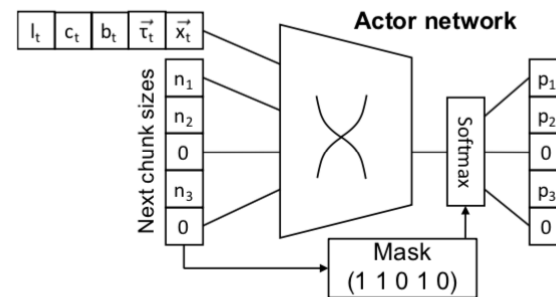


Figure 6: Modification to the state input and the softmax output to support multiple videos.

V. Mnih et al. 2016. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning*. 1928–1937.

Published in Transactions on Machine Learning Research (11/2022)

A Generalist Agent

Scott Reed^{*,†}, Konrad Żołna^{*}, Emilio Parisotto^{*}, Sergio Gómez Colmenarejo[†], Alexander Novikov, Gabriel Barth-Maron, Mai Giménez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, Tom Eccles, Jake Bruce, Ali Razavi, Ashley Edwards, Nicolas Heess, Yutian Chen, Raia Hadsell, Oriol Vinyals, Mahyar Bordbar and Nando de Freitas[†]

^{*}Equal contributions, [†]Equal senior contributions, All authors are affiliated with DeepMind

reedscot@deepmind.com

Reviewed on OpenReview: <https://openreview.net/forum?id=1ikK0kHvj>

Abstract

Inspired by progress in large-scale language modeling, we apply a similar approach towards building a single generalist agent beyond the realm of text outputs. The agent, which we refer to as Gato, works as a multi-modal, multi-task, multi-embodiment generalist policy. The same network with the same weights can play Atari, caption images, chat, stack blocks with a real robot arm and much more, deciding based on its context whether to output text, joint torques, button presses, or other tokens. In this report we describe the model and the data, and document the current capabilities of Gato.



Figure 1: **A generalist agent.** Gato can sense and act with different embodiments across a wide range of environments using a single neural network with the same set of weights. Gato was trained on 604 distinct tasks with varying modalities, observations and action specifications.

一个通用的RL大模型

多模态的数据

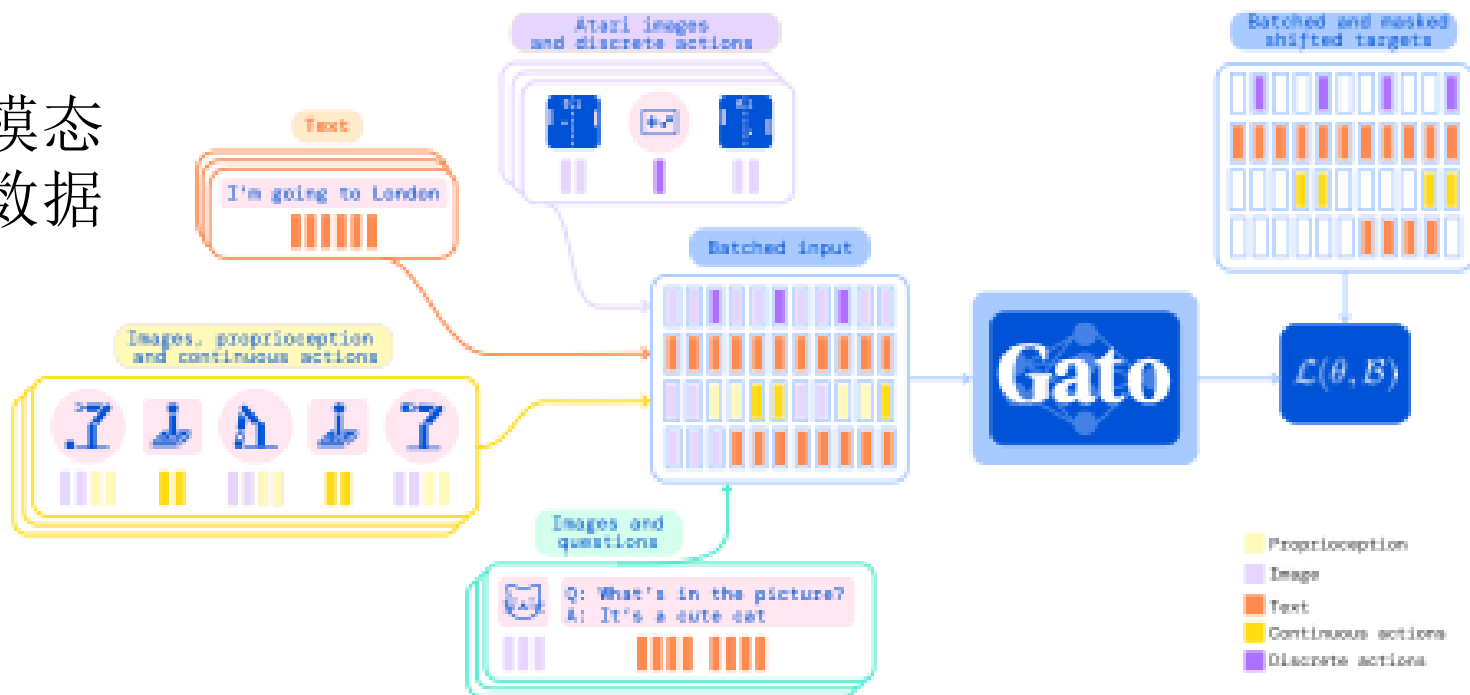


Figure 2: **Training phase of Gato.** Data from different tasks and modalities is serialized into a flat sequence of tokens, batched, and processed by a transformer neural network akin to a large language model. Masking is used such that the loss function is applied only to target outputs, i.e. text and various actions.

Training language models to follow instructions with human feedback

Long Ouyang* Jeff Wu* Xu Jiang* Diogo Almeida* Carroll L. Wainwright*

Pamela Mishkin* Chong Zhang Sandhini Agarwal Katarina Slama Alex Ray

John Schulman Jacob Hilton Fraser Kelton Luke Miller Maddie Simens

Amanda Askell†

Peter Welinder

Paul Christiano*†

Jan Leike*

Ryan Lowe*

OpenAI

Abstract

Making language models bigger does not inherently make them better at following a user's intent. For example, large language models can generate outputs that are untruthful, toxic, or simply not helpful to the user. In other words, these models are not *aligned* with their users. In this paper, we show an avenue for aligning language models with user intent on a wide range of tasks by fine-tuning with human feedback. Starting with a set of labeler-written prompts and prompts submitted through the OpenAI API, we collect a dataset of labeler demonstrations of the desired model behavior, which we use to fine-tune GPT-3 using supervised learning. We then collect a dataset of rankings of model outputs, which we use to further fine-tune this supervised model using reinforcement learning from human feedback. We call the resulting models *InstructGPT*. In human evaluations on our prompt distribution, outputs from the 1.3B parameter InstructGPT model are preferred to outputs from the 175B GPT-3, despite having 100x fewer parameters. Moreover, InstructGPT models show improvements in truthfulness and reductions in toxic output generation while having minimal performance regressions on public NLP datasets. Even though InstructGPT still makes simple mistakes, our results show that fine-tuning with human feedback is a promising direction for aligning language models with human intent.

1 Introduction

Large language models (LMs) can be “prompted” to perform a range of natural language processing (NLP) tasks, given some examples of the task as input. However, these models often express unintended behaviors such as making up facts, generating biased or toxic text, or simply not following user instructions (Bender et al., 2021; Bommasani et al., 2021; Kenton et al., 2021; Weidinger et al., 2021; Tamkin et al., 2021; Gehman et al., 2020). This is because the language modeling objective

*Primary authors. This was a joint project of the OpenAI Alignment team. RL and JL are the team leads. Corresponding author: lowe@openai.com.

†Work done while at OpenAI. Current affiliations: AA: Anthropic; PC: Alignment Research Center.

基于人类反馈的强化学习(RLHF)

■ ChatGPT提升了理解人类思维的准确性的原因在于利用了基于人类反馈数据的系统进行模型训练

■ RL算法: PPO, OpenAI, 2017

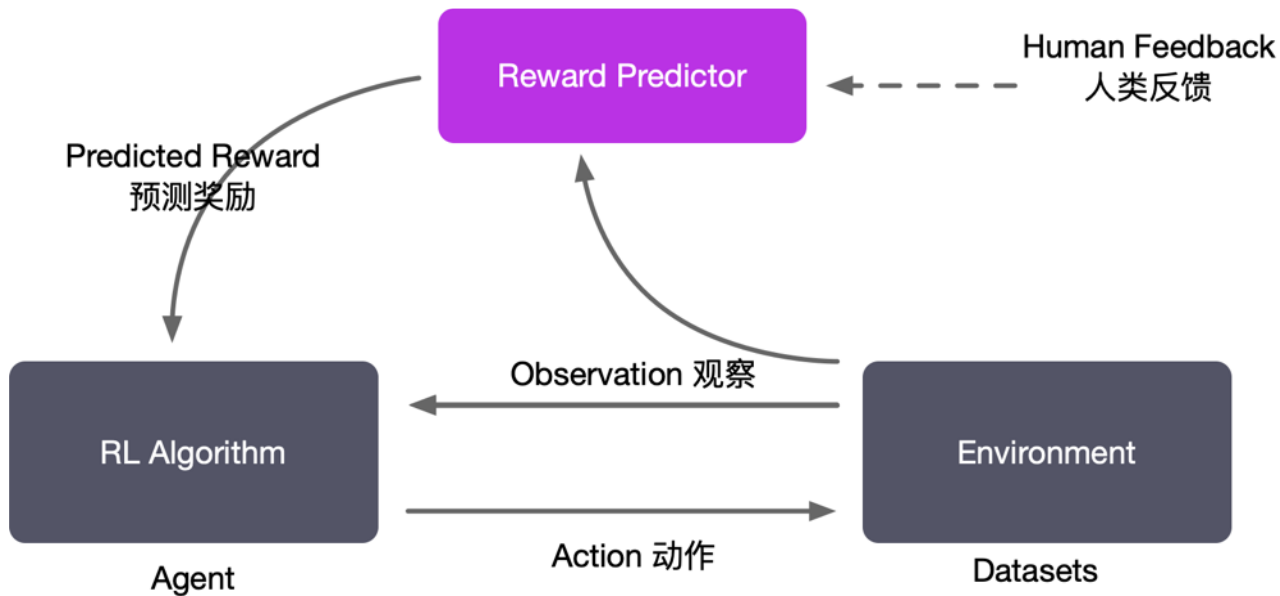
Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI
{joschu, filip, prafulla, alec, oleg}@openai.com

Abstract

We propose a new family of policy gradient methods for reinforcement learning, which alternate between sampling data through interaction with the environment, and optimizing a “surrogate” objective function using stochastic gradient ascent. Whereas standard policy gradient methods perform one gradient update per data sample, we propose a novel objective function that enables multiple epochs of minibatch updates. The new methods, which we call proximal policy optimization (PPO), have some of the benefits of trust region policy optimization (TRPO), but they are much simpler to implement, more general, and have better sample complexity (empirically). Our experiments test PPO on a collection of benchmark tasks, including simulated robotic locomotion and Atari game playing, and we show that PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time.

- RLHF: 通过人类标注数据训练得到 Reward Model (有了人类标注数据, 然后反推人类因为什么样的奖励函数才会采取这些行为), 有了奖励函数之后, 使用一般的强化学习的方法找出最优策略/动作。





- ChatGPT和InstructGPT^[33]在模型结构，训练方式上都基本一致
- 核心技术：指示学习（Instruction Learning）
- 核心技术：人类反馈的强化学习（RLHF）。
- InstructGPT基于GPT-3.5微调得到，并且将RLHF和预训练模型巧妙的结合起来，提升了模型的有用性，真实性和无害性。

- 第一步：这是一个有监督微调（Supervised fine-tuning, SFT）的过程。对于用户提交的提示词（prompt），由人类提供回答，然后把这些成对的提问和回答数据作为训练数据，对GPT-3模型进行微调训练。
- 需要巨大的人力成本。

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

🗣️
Explain the moon landing to a 6 year old

A labeler demonstrates the desired output behavior.

👤
Some people went to the moon...

This data is used to fine-tune GPT-3 with supervised learning.

🧠
SFT
📄📄📄

Submit

Skip

«

Page 3 / 11

»

Total time: 05:39

Instruction

Summarize the following news article:

====
{article}
=====

Include output

Output A

summary1

Rating (1 = worst, 7 = best)

1

2

3

4

5

6

7

Fails to follow the correct instruction / task ? ☐ Yes ☐ No

Inappropriate for customer assistant ? ☐ Yes ☐ No

Contains sexual content ☐ Yes ☐ No

Contains violent content ☐ Yes ☐ No

Encourages or fails to discourage violence/abuse/terrorism/self-harm ☐ Yes ☐ No

Denigrates a protected class ☐ Yes ☐ No

Gives harmful advice ? ☐ Yes ☐ No

Expresses moral judgment ☐ Yes ☐ No

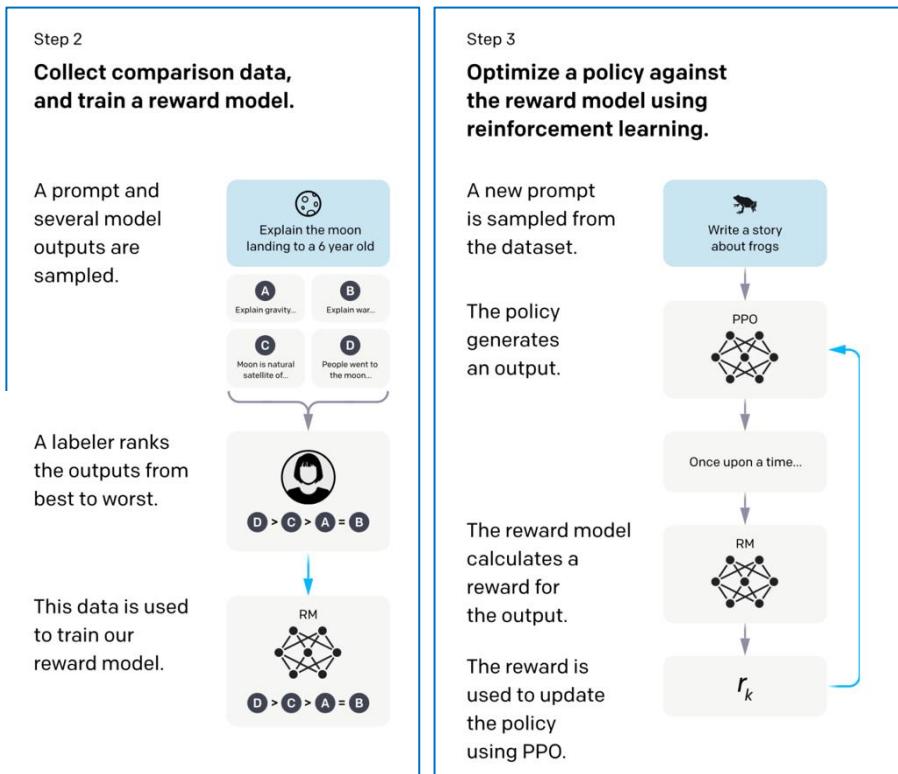
Notes

(Optional) notes

可参考OpenAI 联合创始人 John Schulman 视频:

https://www.bilibili.com/video/BV1SL41167c5/?spm_id_from=333.337.search-card.all.click&vd_source=6cbf6b9c8277ed78e236c5c983aec4f8

- **第二步**: 对于一个提示, SFT模型输出几个不同的回答, 人类标记员则需要给这些回答**打分** (从最理想到最不理想)。
- 通过这些标注, 训练出一个可以捕捉人类偏好的**奖励模型** (Reward model, RM)。
- 相比于人为写出回答, 对模型的回答进行打分要容易的多, 这大大降低了人力成本, 从而可以获得更加庞大的监督数据。
- **第三步**: 将GPT-3模型视为一个**策略网络**, 利用强化学习近端策略优化 (PPO) 算法生成输出, 并使用第二步得到的RM模型对输出打分, 从而更新模型。
- 通过迭代步骤2和3, 不断优化GPT-3模型, 最后实现和人类偏好的对齐。
- 这里, ChatGPT是一个生成器, RM是一个判别器, ChatGPT试图愚弄RM, 而RM在人类的帮助下, 能学习和分辨符合人类认知的内容。当RM不能再分辨时, 模型就会收敛。



图片: InstructGPT流程图